

Option Trading

Session Six: Creating an Options Backtester

Session Five Overview

- Hedging in practice
- Expiration trading
- Early exercise
- Trade planning and evaluation
- Risk measures
- An example that summarises the volatility trading process

Session Six Overview

- Overview of Backtesting
- Key Steps in Options Backtesting
- Structure of an Options Backtester
- Building a Basic Backtester for Naked Options
- Expanding the Backtester: Handling Multi-Leg Strategies

Overview of Backtesting

Backtesting

- Evaluates a strategy using historical data.
- Simulates real-world trading conditions.
- Key metrics: total return, Sharpe ratio, drawdown.
- Validates strategy performance before live trading.
- Limitations: Overfitting and reliance on past data.

Key Steps in Options Backtesting

Key Steps

1. Collecting options and underlying data
2. Data structuring, cleaning
3. Classifying data as per expiry date
4. Filtering options contracts based on expiry and liquidity
5. Simulating trading execution (Event-based simulation)
6. Performance metrics for backtesting

1. Collecting Options and Underlying Data

How Options Data Differs from Underlying Data?

Underlying Data of S&P 500 Index on '2023-01-04'

	Open	High	Low	Close	Volume
Date					
2023-01-04	3840.36	3873.16	3815.77	3852.97	4414080000

1. Collecting Options and Underlying Data

How Options Data Differs from Underlying Data?

Options Data of SPX Index on '2023-01-04'

	symbol	expire_date	dte	c_delta	c_gamma	c_vega	c_theta	c_rho	c_iv	c_volume	...	p_delta	p_gamma	p_vega	p_theta
quote_date															
2023-01-04	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	1.0	...	0.00000	0.0	0.00023	-0.02481
2023-01-04	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	1.0	...	0.00000	0.0	0.00044	-0.02531
2023-01-04	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	NaN	...	-0.00025	0.0	0.00059	-0.02508
2023-01-04	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	1.0	...	0.00000	0.0	0.00114	-0.02538
2023-01-04	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	NaN	...	0.00000	0.0	0.00100	-0.02550
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2023-01-04	SPX	2027-12-17	1808.0	0.05301	0.00002	9.13213	-0.05543	8.72598	0.17269	1.0	...	-1.00000	0.0	0.00000	0.00000
2023-01-04	SPX	2027-12-17	1808.0	0.03950	0.00007	7.28376	-0.04336	6.61895	0.17094	10.0	...	-1.00000	0.0	0.00000	0.00000
2023-01-04	SPX	2027-12-17	1808.0	0.03288	0.00008	6.25757	-0.03781	5.49983	0.17239	NaN	...	-1.00000	0.0	0.00000	0.00000
2023-01-04	SPX	2027-12-17	1808.0	0.02790	0.00008	5.43427	-0.03246	4.63926	0.17480	1.0	...	-1.00000	0.0	0.00000	0.00000
2023-01-04	SPX	2027-12-17	1808.0	0.01331	0.00001	2.95223	-0.01860	2.25789	0.18573	1.0	...	-1.00000	0.0	0.00000	0.00000

7549 rows × 29 columns

Options data of multiple expiry and multiple strike prices

1. Collecting Options and Underlying Data

Collecting options and underlying data

- Brokers, data providers
- APIs, data downloads
- Free data, paid data

In this lecture:

optionsDX: Free Historical Options Trading Data

Implementation: Collecting Options and Underlying Data

Output

	[QUOTE_UNIXTIME]	[QUOTE_READTIME]	[QUOTE_DATE]	[QUOTE_TIME_HOURS]	[UNDERLYING_LAST]	[EXPIRE_DATE]	[EXPIRE_UNIX]	[DTE]
0	1672866000	2023-01-04 16:00	2023-01-04	16.0	3853.39	2023-01-04	1672866000	0.0
1	1672866000	2023-01-04 16:00	2023-01-04	16.0	3853.39	2023-01-04	1672866000	0.0
2	1672866000	2023-01-04 16:00	2023-01-04	16.0	3853.39	2023-01-04	1672866000	0.0
3	1672866000	2023-01-04 16:00	2023-01-04	16.0	3853.39	2023-01-04	1672866000	0.0
4	1672866000	2023-01-04 16:00	2023-01-04	16.0	3853.39	2023-01-04	1672866000	0.0
...	...	...	...	...	...	...	...	...
2009078	1703883600	2023-12-29 16:00	2023-12-29	16.0	4772.17	2029-12-21	1892581200	2184.0
2009079	1703883600	2023-12-29 16:00	2023-12-29	16.0	4772.17	2029-12-21	1892581200	2184.0
2009080	1703883600	2023-12-29 16:00	2023-12-29	16.0	4772.17	2029-12-21	1892581200	2184.0
2009081	1703883600	2023-12-29 16:00	2023-12-29	16.0	4772.17	2029-12-21	1892581200	2184.0
2009082	1703883600	2023-12-29 16:00	2023-12-29	16.0	4772.17	2029-12-21	1892581200	2184.0

2009083 rows × 33 columns

2. Data Structure & Cleaning

Structure of Options Data

- **Underlying Asset:** Asset the options are based on.
- **Strike Price:** Price at which the option is exercised.
- **Expiration Date:** Date when the option expires.
- **Option Type:** Call or Put.
- **Bid/Ask Price:** Highest bid and lowest ask.
- **Volume/Open Interest:** Volume of contracts traded, open contracts.
- **Implied Volatility (IV):** Market's volatility expectation.
- **Option Greeks:** Delta, Gamma, Theta, Vega, Rho (sensitivity to price, time, volatility).

2. Data Structure & Cleaning

Cleaning Options Data

1. Columns:
 - a. Rename columns if needed
 - b. Remove unnecessary column data
 - c. Add required columns
2. Clean unnecessary space in data
3. Check the data types and correct

2. Data Structure & Cleaning

Cleaning Options Data

Rename columns if needed

```
Index(['[QUOTE_UNIXTIME]', '[QUOTE_READTIME]', '[QUOTE_DATE]',  
      '[QUOTE_TIME_HOURS]', '[UNDERLYING_LAST]', '[EXPIRE_DATE]',  
      '[EXPIRE_UNIX]', '[DTE]', '[C_DELTA]', '[C_GAMMA]', '[C_VEGA]',  
      '[C_THETA]', '[C_RHO]', '[C_IV]', '[C_VOLUME]', '[C_LAST]',  
      '[C_SIZE]', '[C_BID]', '[C_ASK]', '[STRIKE]', '[P_BID]',  
      '[P_ASK]', '[P_SIZE]', '[P_LAST]', '[P_DELTA]', '[P_GAMMA]',  
      '[P_VEGA]', '[P_THETA]', '[P_RHO]', '[P_IV]', '[P_VOLUME]',  
      '[STRIKE_DISTANCE]', '[STRIKE_DISTANCE_PCT]'],  
      dtype='object')
```

- Unnecessary leading space, square brackets and capitalisation

2. Data Structure & Cleaning

Cleaning Options Data

Rename columns if needed

```
Index(['[QUOTE_UNIXTIME]', '[QUOTE_READTIME]', '[QUOTE_DATE]',  
      '[QUOTE_TIME_HOURS]', '[UNDERLYING_LAST]', '[EXPIRE_DATE]',  
      '[EXPIRE_UNIX]', '[DTE]', '[C_DELTA]', '[C_GAMMA]', '[C_VEGA]',  
      '[C_THETA]', '[C_RHO]', '[C_IV]', '[C_VOLUME]', '[C_LAST]',  
      '[C_SIZE]', '[C_BID]', '[C_ASK]', '[STRIKE]', '[P_BID]',  
      '[P_ASK]', '[P_SIZE]', '[P_LAST]', '[P_DELTA]', '[P_GAMMA]',  
      '[P_VEGA]', '[P_THETA]', '[P_RHO]', '[P_IV]', '[P_VOLUME]',  
      '[STRIKE_DISTANCE]', '[STRIKE_DISTANCE_PCT]'],  
      dtype='object')
```



```
Index(['quote_unixtime', 'quote_readtime', 'quote_date', 'quote_time_hours',  
      'underlying_last', 'expire_date', 'expire_unix', 'dte', 'c_delta',  
      'c_gamma', 'c_vega', 'c_theta', 'c_rho', 'c_iv', 'c_volume', 'c_last',  
      'c_size', 'c_bid', 'c_ask', 'strike', 'p_bid', 'p_ask', 'p_size',  
      'p_last', 'p_delta', 'p_gamma', 'p_vega', 'p_theta', 'p_rho', 'p_iv',  
      'p_volume', 'strike_distance', 'strike_distance_pct'],  
      dtype='object')
```

2. Data Structure & Cleaning

Cleaning Options Data

Remove unnecessary column data

```
Index(['quote_unixtime', 'quote_readtime', 'quote_date', 'quote_time_hours',  
      'underlying_last', 'expire_date', 'expire_unix', 'dte', 'c_delta',  
      'c_gamma', 'c_vega', 'c_theta', 'c_rho', 'c_iv', 'c_volume', 'c_last',  
      'c_size', 'c_bid', 'c_ask', 'strike', 'p_bid', 'p_ask', 'p_size',  
      'p_last', 'p_delta', 'p_gamma', 'p_vega', 'p_theta', 'p_rho', 'p_iv',  
      'p_volume', 'strike_distance', 'strike_distance_pct'],  
      dtype='object')
```

2. Data Structure & Cleaning

Cleaning Options Data

Add required columns

- Can add column 'symbol' to represent the SPX contracts

2. Data Structure & Cleaning

Cleaning Options Data

Clean unnecessary space in data

' 0.161640 ' → '0.161640 '

2. Data Structure & Cleaning

Cleaning Options Data: Check the data types and

#	Column	Non-Null Count	Dtype
0	underlying_last	547280 non-null	float64
1	symbol	547280 non-null	object
2	expire_date	547280 non-null	object
3	dte	547280 non-null	float64
4	c_delta	547280 non-null	float64
5	c_gamma	547280 non-null	float64
6	c_vega	547280 non-null	float64
7	c_theta	547280 non-null	float64
8	c_rho	547280 non-null	float64
9	c_iv	547280 non-null	object
10	c_volume	547280 non-null	object
11	c_last	547280 non-null	float64
12	c_size	547280 non-null	object
13	c_bid	547280 non-null	float64
14	c_ask	547280 non-null	float64
15	strike	547280 non-null	float64
16	p_bid	547280 non-null	float64
17	p_ask	547280 non-null	float64
18	p_size	547280 non-null	object
19	p_last	547280 non-null	float64
20	p_delta	547280 non-null	float64
21	p_gamma	547280 non-null	float64
22	p_vega	547280 non-null	float64
23	p_theta	547280 non-null	float64
24	p_rho	547280 non-null	float64
25	p_iv	547280 non-null	object
26	p_volume	547280 non-null	object
27	strike_distance	547280 non-null	float64
28	strike_distance_pct	547280 non-null	float64

#	Column	Non-Null Count	Dtype
0	underlying_last	107339 non-null	float64
1	symbol	107339 non-null	object
2	expire_date	107339 non-null	datetime64[ns]
3	dte	107339 non-null	float64
4	c_delta	107339 non-null	float64
5	c_gamma	107339 non-null	float64
6	c_vega	107339 non-null	float64
7	c_theta	107339 non-null	float64
8	c_rho	107339 non-null	float64
9	c_iv	107339 non-null	float64
10	c_volume	107339 non-null	float64
11	c_last	107339 non-null	float64
12	c_size	107339 non-null	object
13	c_bid	107339 non-null	float64
14	c_ask	107339 non-null	float64
15	strike	107339 non-null	float64
16	p_bid	107339 non-null	float64
17	p_ask	107339 non-null	float64
18	p_size	107339 non-null	object
19	p_last	107339 non-null	float64
20	p_delta	107339 non-null	float64
21	p_gamma	107339 non-null	float64
22	p_vega	107339 non-null	float64
23	p_theta	107339 non-null	float64
24	p_rho	107339 non-null	float64
25	p_iv	107339 non-null	float64
26	p_volume	107339 non-null	float64
27	strike_distance	107339 non-null	float64
28	strike_distance_pct	107339 non-null	float64

3. Classifying data as per expiry date

Categorize options expiry dates based on different expiry types (LEAPS, Monthly, Weekly, etc.) helps in strategy development and analysis.

3. Classifying data as per expiry date

Classification Criteria:

- **LEAPS:** Expiry in 365 days or more.
- **Monthly Expirations:** Third Friday of the month (Near, Next, Far).
- **Weekly Expirations:** Any Friday that is NOT the third Friday.
- **End-of-Month:** Last trading day of the month.
- **Daily Expirations:** Default for any other scenarios.

3. Classifying data as per expiry date

Expiry Classification Logic:

- **LEAPS:** Expirations 1 year or more.
- **Monthly Expiry:**
 - Near Month: Current month's third Friday.
 - Next Month: Next month's third Friday.
 - Far Month: The third Friday two months out.
 - Later Month: Any other monthly expiry.
- **Weekly Expiry:** Any Friday that isn't the third Friday.
- **End-of-Month:** Last trading day of the month.
- **Daily Expiry:** Default classification for daily expirations.

Python Implementation

Data structuring and cleaning
Classifying data as per expiry date

3. Classifying data as per expiry date

quote_date	underlying_last	symbol	expire_date	dte	c_delta	c_gamma	c_vega	c_theta	c_rho	c_iv	...	p_last	p_delta	p_gamma	p_vega
2023-01-04	3853.39	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	...	0.03	0.00000	0.0	0.00023
2023-01-04	3853.39	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	...	0.00	0.00000	0.0	0.00044
2023-01-04	3853.39	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	...	0.00	-0.00025	0.0	0.00059
2023-01-04	3853.39	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	...	0.00	0.00000	0.0	0.00114
2023-01-04	3853.39	SPX	2023-01-04	0.0	1.00000	0.00000	0.00000	0.00000	0.00000	NaN	...	0.00	0.00000	0.0	0.00100
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.29633	0.00012	36.63114	-0.19956	65.38557	0.17401	...	0.00	-1.00000	0.0	0.00000
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.27129	0.00010	35.36834	-0.18921	60.66536	0.17063	...	0.00	-1.00000	0.0	0.00000
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.24709	0.00014	33.93840	-0.17756	55.97923	0.16688	...	0.00	-1.00000	0.0	0.00000
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.22447	0.00015	32.37197	-0.16644	51.39303	0.16395	...	0.00	-1.00000	0.0	0.00000
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.20224	0.00010	30.71374	-0.15519	46.99074	0.16164	...	0.00	-1.00000	0.0	0.00000

2009083 rows × 29 columns

```
options_data_raw.expiry_type.unique()
```

```
array(['Daily', 'Monthly (Later Month)', 'Near Month', 'End-of-Month',  
      'Next Month', 'LEAPS', 'Far Month'], dtype=object)
```

4. Filtering options contracts based on expiry and liquidity

1. Filter active contracts
2. Filter relevant contracts
3. Remove illiquid contracts
4. Filter based on IV
5. Filter based on bid and ask values

Python Implementation

Filtering options contracts based on expiry
and liquidity

Output

quote_date	underlying_last	symbol	expire_date	dte	c_delta	c_gamma	c_vega	c_theta	c_rho	c_iv	...	p_delta	p_gamma	p_vega	p_theta
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94620	0.00070	0.27153	-1.25966	0.05946	0.44352	...	-0.00387	0.00017	0.02811	-0.07474
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.92149	0.00241	0.38253	-1.56025	0.08911	0.24565	...	-0.02479	0.00134	0.15614	-0.41570
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94186	0.00440	0.29954	-0.46034	0.06840	0.14069	...	-0.10746	0.00433	0.49274	-1.92152
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.90582	0.00535	0.43862	-1.06060	0.09031	0.14705	...	-0.13617	0.00508	0.58092	-2.50061
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.88661	0.00617	0.50476	-1.30830	0.09619	0.14224	...	-0.16595	0.00565	0.66387	-3.08991
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2023-12-29	4772.17	SPX	2028-12-15	1813.0	0.63305	0.00012	32.01432	-0.26092	86.95073	0.24131	...	-0.31767	0.00026	33.43123	-0.08232
2023-12-29	4772.17	SPX	2028-12-15	1813.0	0.62098	0.00011	32.63014	-0.26413	87.23759	0.23697	...	-0.34102	0.00027	34.11695	-0.08228
2023-12-29	4772.17	SPX	2028-12-15	1813.0	0.60771	0.00012	33.21342	-0.26623	87.62850	0.23096	...	-0.36721	0.00030	34.61398	-0.07869
2023-12-29	4772.17	SPX	2028-12-15	1813.0	0.58014	0.00012	34.31982	-0.27030	87.26228	0.22195	...	-0.42879	0.00028	34.99292	-0.06812
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.58848	0.00008	36.20196	-0.23995	95.09132	0.24139	...	-0.41479	0.00032	37.26064	-0.05655

417212 rows x 30 columns

quote_date	underlying_last	symbol	expire_date	dte	c_delta	c_gamma	c_vega	c_theta	c_rho	c_iv	...	p_delta	p_gamma	p_vega	p_theta
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94891	0.00052	0.26077	-1.15966	0.05262	0.49309	...	-0.00325	0.00013	0.02468	-0.07464
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94730	0.00059	0.26767	-1.26023	0.05664	0.47499	...	-0.00361	0.00013	0.02605	-0.07488
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94620	0.00070	0.27153	-1.25966	0.05946	0.44352	...	-0.00387	0.00017	0.02811	-0.07474
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.92149	0.00241	0.38253	-1.56025	0.08911	0.24565	...	-0.02479	0.00134	0.15614	-0.41570
2023-01-04	3853.39	SPX	2023-01-05	1.0	0.94186	0.00440	0.29954	-0.46034	0.06840	0.14069	...	-0.10746	0.00433	0.49274	-1.92152
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2023-12-29	4772.17	SPX	2028-12-15	1813.0	0.52223	0.00012	36.06815	-0.27288	84.23397	0.20778	...	-0.62572	0.00054	29.80007	-0.03667
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.90700	0.00005	11.61324	-0.07469	26.20164	0.41725	...	-0.02609	0.00005	6.85673	-0.03329
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.69773	0.00006	30.19011	-0.20761	85.71353	0.28295	...	-0.20537	0.00009	30.05622	-0.07625
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.58848	0.00008	36.20196	-0.23995	95.09132	0.24139	...	-0.41479	0.00032	37.26064	-0.05655
2023-12-29	4772.17	SPX	2029-12-21	2184.0	0.51373	0.00016	38.65328	-0.24638	93.49492	0.21787	...	-0.89490	0.00096	11.09994	-0.00463

783101 rows x 30 columns

5. Simulating trading execution (Event-based backtest)

What is Event-Based Backtest?

A trading execution model where actions are triggered by specific events (e.g., price changes, volume thresholds, or market conditions).

5. Simulating trading execution (Event-based backtest)

Event Triggers: Market data or price movements (e.g., price threshold, new order).

Execution Logic: Orders placed based on predefined rules when events occur.

Time Sensitivity: Events processed in real-time, simulating actual market trades.

5. Simulating trading execution (Event-based backtest)

Advantages:

- More realistic representation of real-world trading.
- Accounts for market dynamics, reducing the need for overly granular time intervals.
- Optimizes performance by focusing on significant market events rather than every tick or minute.

5. Simulating trading execution (Event-based backtest)

Functions helpful in creating backtester

1. Function to find the strike price of contract based on strike price and expiry date
2. Function to classify data as per exit condition
3. Function to setup options strategy
4. Function to calculate mark to market of trades

Python Implementation

Flow of Backtester and Functions Used

5. Simulating trading execution (Event-based backtest)

Strategy 1: Long Call

Contract: SPX

Expiry: Near month

Entry: Buy ATM Call on first trading day of the month

Exit: Close positions one-day before the expiry

5. Simulating trading execution (Event-based backtest)

How to Consider Trading Costs and Slippages?

Components of Trading Costs:

- **Bid-Ask Spread:** Wider spreads → Higher costs, esp. in volatility.
- **Commissions & Fees:** Broker-dependent, includes exchange & regulatory fees.
- **Market Impact:** Large orders can shift prices, raising costs.

5. Simulating trading execution (Event-based backtest)

Understanding Slippage:

Slippage: Difference between expected & actual execution price.

Causes: Volatility, execution speed, liquidity constraints.

How to Consider Trading Costs and Slippages?

Consider a multiple of trade turnover to represent trading costs and slippages while backtesting

Python Implementation

Creating Backtester for Long Call Strategy

Output

Trade No: 1

Entry:

- Date: 2023-01-04 00:00:00
- Premium Paid: 59.6

Exit:

- Type: Expiry
- Date: 2023-01-19 00:00:00
- Premium Received: 58.9
- Profit/Loss: -0.7
- Cumulative PnL: -0.7

Trade No: 2

Entry:

- Date: 2023-02-01 00:00:00
- Premium Paid: 64.3

Exit:

- Type: Expiry
- Date: 2023-02-16 00:00:00
- Premium Received: 2.3
- Profit/Loss: -62.0
- Cumulative PnL: -62.7

Trade No: 3

Entry:

- Date: 2023-03-01 00:00:00
- Premium Paid: 83.5

Exit:

- Type: Expiry
- Date: 2023-03-23 00:00:00
- Premium Received: 24.1
- Profit/Loss: -59.4
- Cumulative PnL: -122.1

Trade No: 4

Entry:

- Date: 2023-04-03 00:00:00
- Premium Paid: 59.5

Exit:

- Type: Expiry
- Date: 2023-04-20 00:00:00
- Premium Received: 8.2
- Profit/Loss: -51.3
- Cumulative PnL: -173.4

Trade No: 5

Entry:

- Date: 2023-05-01 00:00:00
- Premium Paid: 55.3

Exit:

- Type: Expiry
- Date: 2023-05-18 00:00:00
- Premium Received: 35.1
- Profit/Loss: -20.2
- Cumulative PnL: -193.6

Trade No: 6

Entry:

- Date: 2023-06-01 00:00:00
- Premium Paid: 56.4

Exit:

- Type: Expiry
- Date: 2023-06-22 00:00:00
- Premium Received: 176.2
- Profit/Loss: 119.8
- Cumulative PnL: -73.8

Trade No: 7

Entry:

- Date: 2023-07-03 00:00:00
- Premium Paid: 44.3

Exit:

- Type: Expiry
- Date: 2023-07-20 00:00:00
- Premium Received: 96.6
- Profit/Loss: 52.3
- Cumulative PnL: -21.5

Trade No: 8

Entry:

- Date: 2023-08-01 00:00:00
- Premium Paid: 49.6

Exit:

- Type: Expiry
- Date: 2023-08-17 00:00:00
- Premium Received: 0.1
- Profit/Loss: -49.5
- Cumulative PnL: -71.0

Trade No: 9

Entry:

- Date: 2023-09-01 00:00:00
- Premium Paid: 50.7

Exit:

- Type: Expiry
- Date: 2023-09-21 00:00:00
- Premium Received: 0.1
- Profit/Loss: -50.6
- Cumulative PnL: -121.6

Trade No: 10

Entry:

- Date: 2023-10-02 00:00:00
- Premium Paid: 55.7

Exit:

- Type: Expiry
- Date: 2023-10-19 00:00:00
- Premium Received: 7.9
- Profit/Loss: -47.8
- Cumulative PnL: -169.4

Trade No: 11

Entry:

- Date: 2023-11-01 00:00:00
- Premium Paid: 47.1

Exit:

- Type: Expiry
- Date: 2023-11-16 00:00:00
- Premium Received: 265.1
- Profit/Loss: 218.0
- Cumulative PnL: 48.6

Trade No: 12

Entry:

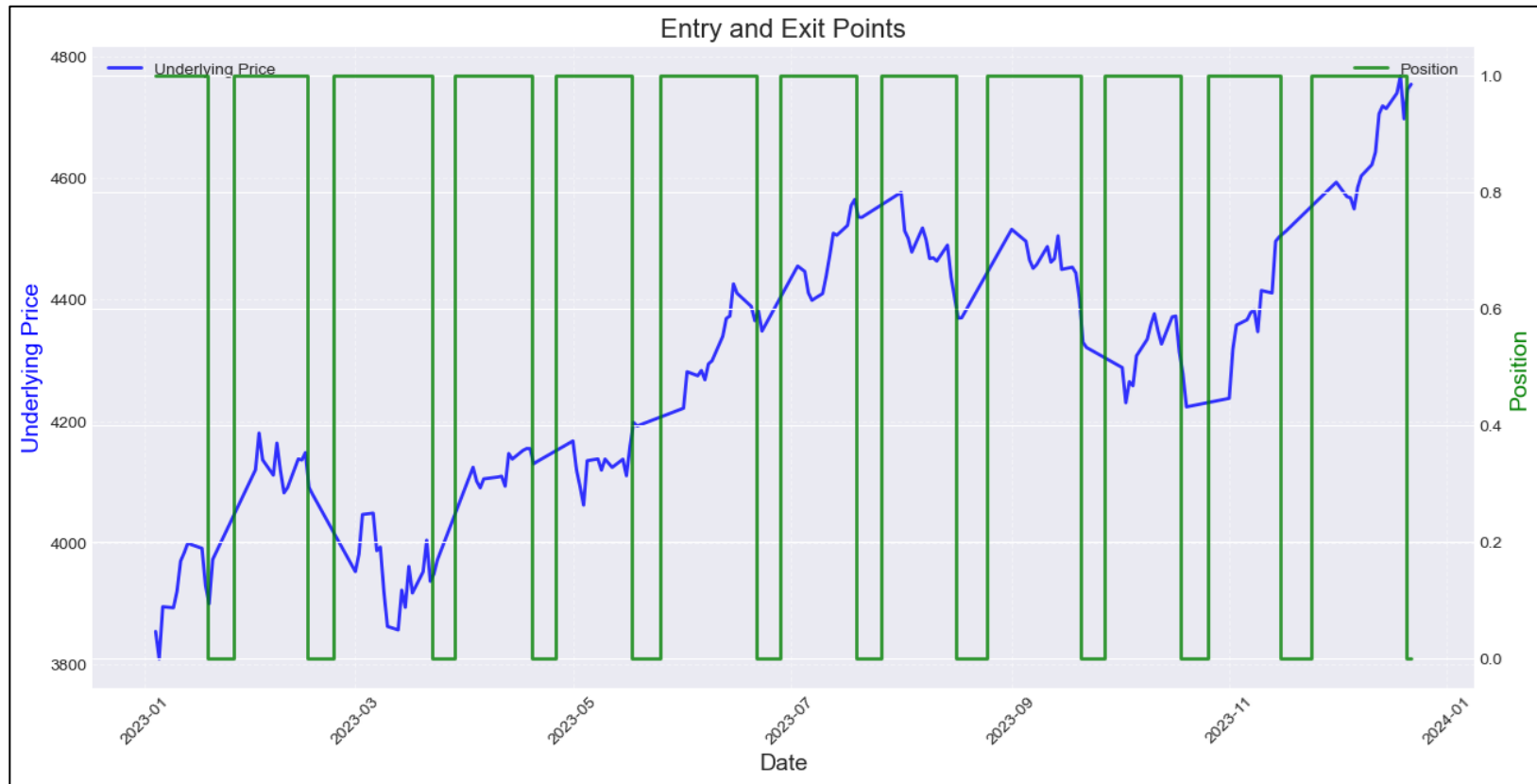
- Date: 2023-12-01 00:00:00
- Premium Paid: 51.8

Exit:

- Type: Expiry
- Date: 2023-12-21 00:00:00
- Premium Received: 127.5
- Profit/Loss: 75.7
- Cumulative PnL: 124.3

Output

Entry and Exit



Output

Trade Sheet

	entry_date	exit_date	exit_type	pnl	turnover	pnl_post_trading_costs_slippages
0	2023-01-04	2023-01-19	Expiry	-37.0	5923.0	-96.230
1	2023-02-01	2023-02-16	Expiry	-3099.5	3330.5	-3132.805
2	2023-03-01	2023-03-23	Expiry	-2972.0	5378.0	-3025.780
3	2023-04-03	2023-04-20	Expiry	-2567.0	3385.0	-2600.850
4	2023-05-01	2023-05-18	Expiry	-1011.5	4518.5	-1056.685
5	2023-06-01	2023-06-22	Expiry	5988.0	11628.0	5871.720
6	2023-07-03	2023-07-20	Expiry	2611.0	7044.0	2540.560
7	2023-08-01	2023-08-17	Expiry	-2475.0	2485.0	-2499.850
8	2023-09-01	2023-09-21	Expiry	-2532.0	2539.0	-2557.390
9	2023-10-02	2023-10-19	Expiry	-2388.5	3178.5	-2420.285
10	2023-11-01	2023-11-16	Expiry	10901.5	15611.5	10745.385
11	2023-12-01	2023-12-21	Expiry	3785.0	8965.0	3695.350

5. Simulating trading execution (Event-based backtest)

Strategy 2: Long Strangle

Contract: SPX

Expiry: Near month

Entry: Buy OTM Call and Put on first trading day of the month

Exit: Close positions one-day before the expiry

Python Implementation

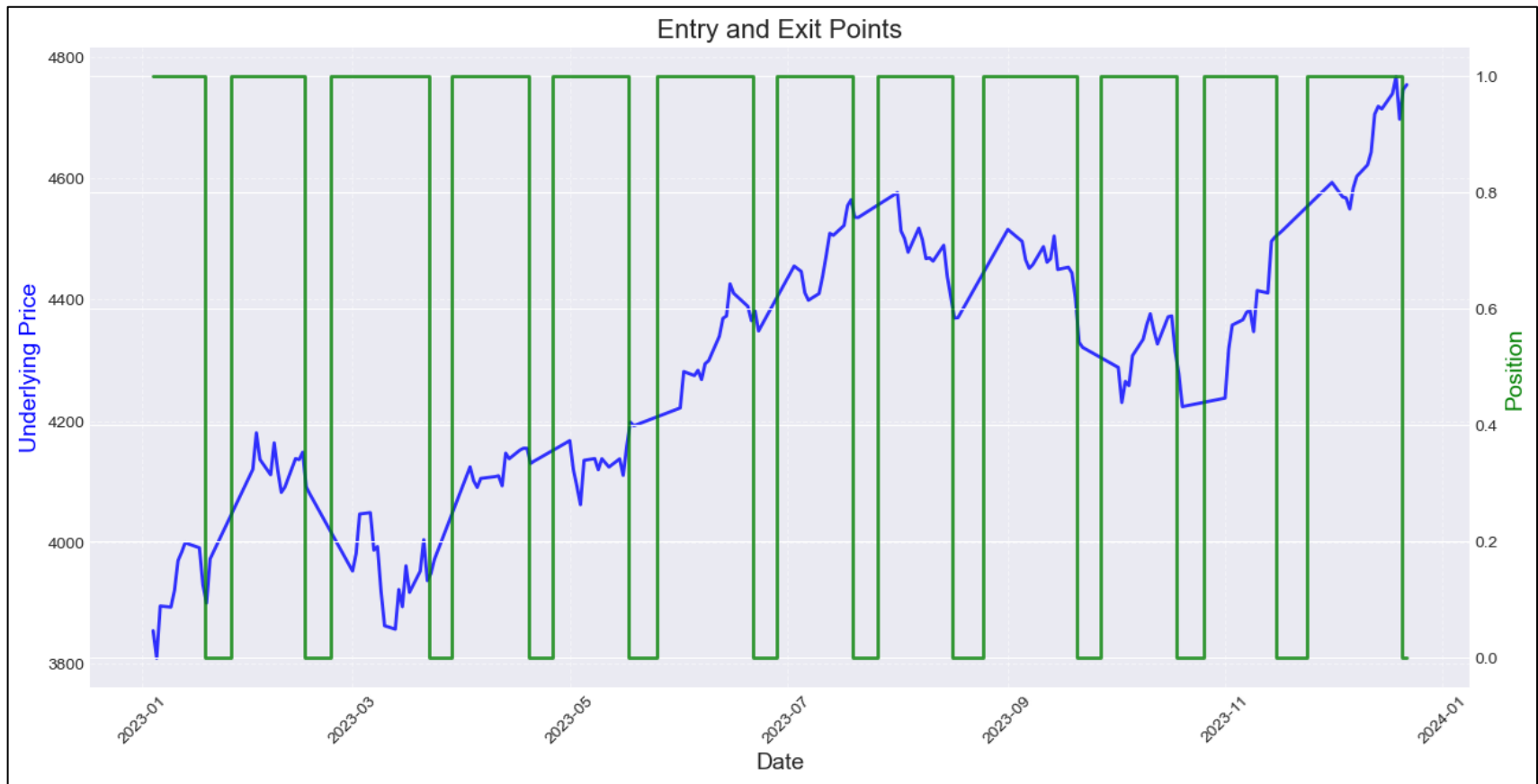
Creating Backtester for Long Strangle Strategy

Output

Trade No: 1 Entry: → Date: 2023-01-04 00:00:00 → Premium Paid: 132.5 Exit: → Type: Expiry → Date: 2023-01-19 00:00:00 → Premium Received: 61.1 → Profit/Loss: -71.4 → Cumulative PnL: -71.4	Trade No: 4 Entry: → Date: 2023-04-03 00:00:00 → Premium Paid: 121.1 Exit: → Type: Expiry → Date: 2023-04-20 00:00:00 → Premium Received: 21.1 → Profit/Loss: -100.0 → Cumulative PnL: -387.2	Trade No: 7 Entry: → Date: 2023-07-03 00:00:00 → Premium Paid: 90.8 Exit: → Type: Expiry → Date: 2023-07-20 00:00:00 → Premium Received: 112.5 → Profit/Loss: 21.7 → Cumulative PnL: -383.8	Trade No: 10 Entry: → Date: 2023-10-02 00:00:00 → Premium Paid: 126.3 Exit: → Type: Expiry → Date: 2023-10-19 00:00:00 → Premium Received: 30.6 → Profit/Loss: -95.7 → Cumulative PnL: -321.2
Trade No: 2 Entry: → Date: 2023-02-01 00:00:00 → Premium Paid: 96.8 Exit: → Type: Expiry → Date: 2023-02-16 00:00:00 → Premium Received: 29.1 → Profit/Loss: -67.7 → Cumulative PnL: -139.1	Trade No: 5 Entry: → Date: 2023-05-01 00:00:00 → Premium Paid: 112.1 Exit: → Type: Expiry → Date: 2023-05-18 00:00:00 → Premium Received: 26.9 → Profit/Loss: -85.2 → Cumulative PnL: -472.4	Trade No: 8 Entry: → Date: 2023-08-01 00:00:00 → Premium Paid: 89.0 Exit: → Type: Expiry → Date: 2023-08-17 00:00:00 → Premium Received: 168.4 → Profit/Loss: 79.4 → Cumulative PnL: -304.4	Trade No: 11 Entry: → Date: 2023-11-01 00:00:00 → Premium Paid: 112.6 Exit: → Type: Expiry → Date: 2023-11-16 00:00:00 → Premium Received: 268.8 → Profit/Loss: 156.2 → Cumulative PnL: -165.0
Trade No: 3 Entry: → Date: 2023-03-01 00:00:00 → Premium Paid: 196.8 Exit: → Type: Expiry → Date: 2023-03-23 00:00:00 → Premium Received: 48.7 → Profit/Loss: -148.1 → Cumulative PnL: -287.2	Trade No: 6 Entry: → Date: 2023-06-01 00:00:00 → Premium Paid: 113.9 Exit: → Type: Expiry → Date: 2023-06-22 00:00:00 → Premium Received: 180.8 → Profit/Loss: 66.9 → Cumulative PnL: -405.5	Trade No: 9 Entry: → Date: 2023-09-01 00:00:00 → Premium Paid: 101.4 Exit: → Type: Expiry → Date: 2023-09-21 00:00:00 → Premium Received: 180.3 → Profit/Loss: 78.9 → Cumulative PnL: -225.5	Trade No: 12 Entry: → Date: 2023-12-01 00:00:00 → Premium Paid: 101.8 Exit: → Type: Expiry → Date: 2023-12-21 00:00:00 → Premium Received: 139.2 → Profit/Loss: 37.4 → Cumulative PnL: -127.6

Output

Entry and Exit



Output

Trade Sheet

	entry_date	exit_date	exit_type	pnl	turnover	pnl_post_trading_costs_slippages
0	2023-01-04	2023-01-19	Expiry	-3574.0	9680.0	-3670.800
1	2023-02-01	2023-02-16	Expiry	-3383.0	6296.0	-3445.960
2	2023-03-01	2023-03-23	Expiry	-7402.5	12274.5	-7525.245
3	2023-04-03	2023-04-20	Expiry	-4999.5	7109.5	-5070.595
4	2023-05-01	2023-05-18	Expiry	-4258.0	6952.0	-4327.520
5	2023-06-01	2023-06-22	Expiry	3343.0	14737.0	3195.630
6	2023-07-03	2023-07-20	Expiry	1082.5	10167.5	980.825
7	2023-08-01	2023-08-17	Expiry	3972.0	12872.0	3843.280
8	2023-09-01	2023-09-21	Expiry	3948.0	14084.0	3807.160
9	2023-10-02	2023-10-19	Expiry	-4784.0	7846.0	-4862.460
10	2023-11-01	2023-11-16	Expiry	7808.5	19068.5	7617.815
11	2023-12-01	2023-12-21	Expiry	1873.5	12051.5	1752.985

5. Simulating trading execution (Event-based backtest)

Exercise: Long Straddle

Contract: SPX

Expiry: Near month

Entry: Buy ATM call and put on first trading day of the month

Exit: Close positions one-day before the expiry

6. Performance metrics for backtesting

Trade Level Analytics:

1. Total profit or loss
2. Total trades
3. Analysis of winners and losing trades
 - a. Profit per trade
 - b. Ratio of winning to losing trades
4. Average holding time
5. Profit Factor

6. Performance metrics for backtesting

Trading Performance Summary

Overall Performance

- Total PnL: \$5,463.14
- Total Trades: 12

Win/Loss Breakdown

- Winning Trades: 4
- Losing Trades: 8
- Win Percentage: 33.33%
- Loss Percentage: 66.67%

Trade Profitability

- Avg Profit per Winning Trade: \$5,821.38
- Avg Loss per Losing Trade: \$2,135.31
- Profit Factor: 1.36

Trade Duration

- Average Holding Time: 18 days

6. Performance metrics for backtesting

Strategy Level Analytics:

1. Incorporating slippages & margins
2. Cumulative returns
3. Sharpe Ratio
4. Annualised Volatility
5. Profit and loss analysis
6. Maximum drawdown

Python Implementation

Performance Analysis of Long Strangle Strategy Backtest

Python Implementation

Making changes to code for strategies that requires shorting

Optimising Strategy Based on Backtest

- Application of risk management
 - Adding stop loss and take profit
 - Position sizing
- Considering additional conditions for entry and exit
 - Considering market condition based filters to improve entry and exit

Python Implementation

Optimising Strategy Based on Backtest

Q&A